COMP 2233: DATA STRUCTURES

Vectors in C++ (Dynamic Arrays)

Standard Template Library (STL) Vector Operations and Memory Management

Assoc.Prof.Dr. Ömer ÇETİN

Problem Solving Pipeline & Software Lifecycle

From Problem Statement to Hardware Results

- Algorithm Development: Human conceptualization of step-by-step logic.
- Program Development: Writing C++ source code utilizing STL containers.
- Program Translation: Compiler translates code and catches syntax errors.
- Program Execution: Hardware executes machine code; catching semantic errors.

1. Algorithm Dev.

Formulating problem logic

2. Program Dev.

Coding C++ source file using <vector>

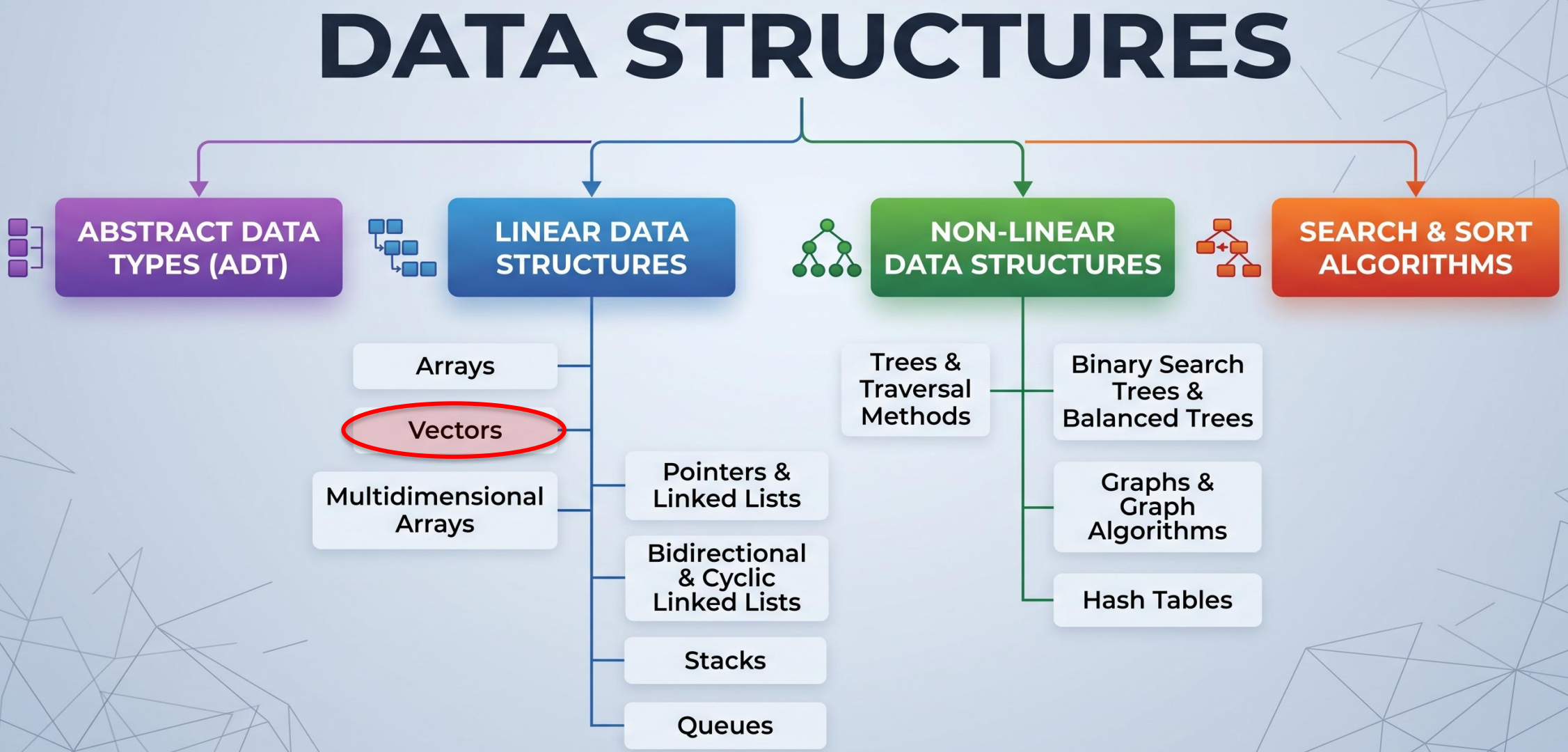
3. Translation

Compiler generates executable binary

4. Execution

CPU executes instructions in hardware

Course Content



What is a Vector? Concept & Key Characteristics

Fundamental Definition

- Dynamic Array: An array container in C++ STL that resizes dynamically at runtime.
- Automatic Memory Management: Automatically allocates and deallocates memory buffer.
- Header Requirement: Included via `#include <vector>` in the C++ Standard Library.
- Unknown Sizes: Ideal when total element count cannot be predicted before launch.

Key Characteristics

- Dynamic Size: Automatically expands capacity when new items are added.
- Contiguous Memory: Elements reside in adjacent RAM slots (cache friendly).
- Random Access O(1): Fast constant-time element lookup via bracket `[]` or `.at()`.
- Flexible Operations: Built-in member methods for insertion, deletion, and search.
- Type-Safe Template: Enforces strict data types (e.g. `vector<int>`, `vector<string>`).

Declaration and Initialization of Vector in C++

Vector Initialization Patterns

- Header Inclusion: `#include <vector>`.
- Empty Declaration: `vector<int> numbers;` (size=0, capacity=0).
- Initializer List (C++11): `vector<int> nums = {10, 20, 30, 40, 50};`.
- Size Method: `nums.size()` returns active element count.
- Expected Output: `10 20 30 40 50`.

```cpp
// C++ Vector Initialization Code
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Declaring empty vector
    vector<int> numbers;

    // Initializing vector with values
    vector<int> nums = {10, 20, 30, 40, 50};

    // Printing elements of vector
    for (size_t i = 0; i < nums.size(); i++) {
        cout << nums[i] << " ";
    }
    cout << endl;
    return 0; // Output: 10 20 30 40 50
}
```

Overview of Common Vector Operations

Core Member Methods in C++ STL Vector

- 1. Adding Elements: `push_back(val)` appends item to end of vector.
- 2. Removing Elements: `pop_back()` removes last element from vector.
- 3. Accessing Elements: `vec[i]` or `vec.at(i)` retrieves item by index.
- 4. Inserting Elements: `vec.insert(pos, val)` inserts item at iterator position.
- 5. Deleting Elements: `vec.erase(pos)` removes item at iterator position.
- 6. Clearing Vector: `vec.clear()` purges all elements from container.
- 7. Checking Size: `vec.size()` returns active element count.

O(1) Tail Mutators

`push_back()`, `pop_back()`

O(1) Accessors

`operator[]`, `.at()`, `.size()`, `.empty()`

O(N) Shift Mutators

`insert()`, `erase()`, `clear()`

1. Adding Elements (`push_back`)

push_back Mechanics

- Functionality: Appends a new value to the very end of the vector.
- Time Complexity: Amortized O(1) constant time.
- Automatic Resizing: If `size == capacity`, vector allocates a new, larger memory block (usually double size) behind the scenes.
- Usage: Primary method for dynamically populating vectors.

```
// push_back Usage Snippet
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> vec;

    vec.push_back(5);  // vec = {5}
    vec.push_back(10); // vec = {5, 10}
    vec.push_back(15); // vec = {5, 10, 15}

    for (int x : vec) {
        cout << x << " ";
    }
    return 0; // Output: 5 10 15
}
```

2. Removing Elements (`pop_back`)

pop_back Mechanics

- Functionality: Destroys and removes the last element from the vector.
- Time Complexity: O(1) constant time (no element shifting required!).
- Capacity Effect: Reduces `size()` by 1 without deallocating allocated `capacity()`.
- Safety Note: Ensure vector is not empty before calling `pop_back()`.

```
// pop_back Usage Snippet
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> vec = {5, 10, 15};

    vec.pop_back(); // Removes last element (15)
    // vec is now {5, 10}

    cout << "Last element: " << vec.back() << endl; // Output: 10
    cout << "New size: " << vec.size() << endl;     // Output: 2
    return 0;
}
```

3. Accessing Elements (`[]` or `at()`)

Bracket Notation vs. `.at()` Method

- Operator `vec[i]`: Fast direct access without boundary check (unsafe for invalid indexes).
- Method `vec.at(i)`: Direct access WITH automatic bounds checking.
- Exception Safety: `.at(i)` throws `std::out_of_range` exception if index is invalid.
- Time Complexity: O(1) constant time for both methods.

```
// Access Methods Comparison Code
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> vec = {5, 10, 15};

    // Fast unchecked access
    cout << "vec[0]: " << vec[0] << endl;   // Output: 5

    // Bounds-checked access
    cout << "vec.at(1): " << vec.at(1) << endl; // Output: 10

    // vec.at(5) would throw std::out_of_range
    return 0;
}
```

4. Inserting Elements (`insert`)

Iterator-Based Insertion Mechanics

- Functionality: Inserts an element at a specified position designated by an iterator.
- Syntax: `vec.insert(vec.begin() + index, value);`.
- Shifting Requirement: Shifts right all subsequent elements.
- Time Complexity: O(N) linear time due to element shifting.

```
// vec.insert Usage Snippet
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> vec = {5, 10};

    // Insert 15 at index 1 (between 5 and 10)
    vec.insert(vec.begin() + 1, 15);
    // vec becomes {5, 15, 10}

    for (int x : vec) cout << x << " ";
    return 0; // Output: 5 15 10
}
```

5. Deleting Elements (`erase`)

Iterator-Based Deletion Mechanics

- Functionality: Removes an element from a specified position designated by an iterator.
- Syntax: `vec.erase(vec.begin() + index);`.
- Shifting Requirement: Shifts left all downstream elements to close the gap.
- Time Complexity: O(N) linear time due to element shifting.

```
// vec.erase Usage Snippet
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> vec = {5, 15, 10};

    // Erase element at index 1 (value 15)
    vec.erase(vec.begin() + 1);
    // vec becomes {5, 10}

    for (int x : vec) cout << x << " ";
    return 0; // Output: 5 10
}
```

6. Clearing the Vector (`clear`)

Purging Vector Contents

- Functionality: Removes all elements from the vector.
- Size Effect: Sets `size()` to 0.
- Capacity Behavior: Does NOT immediately deallocate allocated memory buffer.
- Time Complexity: O(N) linear time to destroy elements.

```
// vec.clear Usage Snippet
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> vec = {5, 10, 15};

    vec.clear(); // Removes all elements

    cout << "Size after clear: " << vec.size() << endl; // Output:
0
    cout << "Is empty? " << (vec.empty() ? "Yes" : "No") << endl;
// Yes
    return 0;
}
```

7. Checking Size & Capacity (`size` vs `capacity`)

Size vs. Capacity Concepts

- `size()`: Returns the exact number of elements currently stored in the vector.
- `capacity()`: Returns total allocated storage space available before re-allocation.
- Relationship: `capacity >= size` always holds.
- Time Complexity: O(1) constant time for both method calls.

```
// size & capacity Snippet
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> vec = {10, 20, 30};

    cout << "Size: " << vec.size() << endl;         // Output: 3
    cout << "Capacity: " << vec.capacity() << endl; // Output: 3
(or >=3)

    vec.push_back(40);
    cout << "New Size: " << vec.size() << endl;     // Output: 4
    cout << "New Cap: " << vec.capacity() << endl;  // Output: 6
(doubled!)
    return 0;
}
```

Advantages of C++ STL Vectors

Architectural Advantages

- Dynamic Resizing: No need to manually specify fixed size or call `malloc`/`free`.
- Built-in Method Suite: Rich API for appending, deleting, sorting, and searching.
- Safe Access: `.at()` provides exception-safe bounds checking.
- RAII Memory Management: Destructors clean up dynamic RAM when vector leaves scope.

Performance Advantages

- Contiguous RAM: Retains spatial cache locality benefits of C static arrays.
- Fast O(1) Access: Instant index-based lookup identical to raw pointers.
- Amortized O(1) Appends: Doubling memory strategy makes `push_back` efficient.
- STL Algorithm Compatibility: Works seamlessly with `std::sort`, `std::find`.

When to Use Vectors vs. Static Arrays

When to Use Vectors

- Unknown Runtime Dataset: Total record volume cannot be predicted at compile time.
- Fluctuating Element Count: Data frequently expands or shrinks during execution.
- Convenience & Safety: Want built-in boundary checking and automatic memory cleanup.
- Modern C++ Standard: Default container choice in modern C++ software engineering.

When to Prefer Static Arrays

- Embedded / Hardware Constraints: Extremely tight RAM limits requiring zero heap overhead.
- Fixed Known Constants: Storing fixed mathematical lookup tables (e.g. 12 months).
- C Interoperability: Interfacing with legacy C C-style APIs requiring raw buffers.

Summary: The Power of C++ STL Vectors

Key Lecture Takeaways

- Versatile & Powerful: Vectors offer the ideal balance between raw performance and dynamic flexibility.
- Contiguous & Dynamic: Maintains O(1) random access while handling automatic capacity doubling.
- Industry Standard: Primary sequential container recommended across modern C++ applications.
- Next Topic: Multi-Dimensional Arrays & Matrix Representations.

Dynamic Memory

Automatic buffer doubling via `push_back()`

O(1) Access

Random lookup via `[]` and exception-safe `.at()`

Memory Safety

Automatic RAII scope cleanup and deallocation